

Лабораторная работа №1. Знакомство с MySQL

Задание 1

Создать базу данных для хранения сведений о результатах сдачи экзаменационной сессии.

Создать в этой базе данных таблицы для следующих сущностей:

- Студент — номер зачетной книжки, ФИО, дата рождения
- Преподаватель — идентификатор, ФИО
- Предмет — идентификатор, название, семестр, преподаватель
- Оценки — связь студент-предмет-оценка

Необходимо:

- выделить сущности
- определить атрибуты
- задать первичные ключи
- задать внешние ключи там, где это необходимо

Задание 2

Необходимо:

- добавить записи с использованием оператора INSERT
- изменить часть данных с использованием оператора UPDATE
- удалить одну из записей с использованием оператора DELETE

Задание 3

Написать запрос, который выводит все предметы.

Задание 4

Написать запрос, который выводит количество студентов.

Задание 5

Написать запрос, который выводит студентов, чья фамилия начинается на заданную подстроку, например Ива , и упорядочивает результат по фамилии и имени.

Задание 6

Написать запрос, который выводит количество студентов по каждому предмету.

Задание 7

Написать запрос, который выводит предметы, по которым количество студентов с оценкой 5 не меньше заданного значения, и упорядочивает результат по названию предмета.

Задание 8

Написать запрос, который выводит студентов, получивших только оценки 4 и 5 по **всем** предметам в указанном семестре.

Лабораторная работа №2. Базовые SQL-запросы на БД sakila

Задание 1

Вывести все платежи с суммой от 10.00 до 11.99 включительно.

В запросе вернуть следующие столбцы:

- идентификатор платежа
- сумма платежа
- дата платежа

Упорядочить результат по сумме по убыванию, затем по дате платежа.

Ожидаемые столбцы

payment_id · amount · payment_date

Задание 2

Вывести все уникальные имена актеров, фамилии которых входят в список (CHASE , CROWE , DAVIS).

Упорядочить результат по имени актера в алфавитном порядке.

Ожидаемые столбцы

first_name

Задание 3

Вывести все фильмы, в которых снимался актер PENELOPE GUINESS .

В запросе вернуть следующие столбцы:

- название фильма
- год выпуска

Ожидаемые столбцы

title · release_year

Задание 4

Для всех покупателей вывести платежи, сумма (amount) которых больше 10.00 .
Если у покупателя нет таких платежей, вместо суммы вывести NULL .

В запросе вернуть следующие столбцы:

- имя покупателя
- фамилия покупателя
- сумма платежа

Ожидаемые столбцы

first_name · last_name · amount

Задание 5

Для каждого магазина вывести суммарную сумму платежей.

В запросе вернуть следующие столбцы:

- идентификатор магазина
- сумма платежей

Ожидаемые столбцы

store_id · amount_sum

Задание 6

Вывести всех актеров, которые снялись более чем в 25 фильмах.

В запросе вернуть следующие столбцы:

- имя актера
- фамилия актера
- количество фильмов

Ожидаемые столбцы

first_name · last_name · films_count

Задание 7

Вывести топ 10 покупателей по общей сумме платежей.

В запросе вернуть следующие столбцы:

- имя покупателя
- фамилия покупателя
- общая сумма платежей

Упорядочить результат по общей сумме платежей по убыванию, затем по фамилии и имени покупателя.

Ожидаемые столбцы

first_name · last_name · amount_sum

Задание 8

Вывести актеров, которые снялись в максимальном количестве фильмов.

В запросе вернуть следующие столбцы:

- имя актера
- фамилия актера
- количество фильмов

Ожидаемые столбцы

first_name · last_name · films_count

Задание 9

Вывести категории фильмов, для которых количество аренд больше 60 .

В запросе вернуть следующие столбцы:

- жанр
- количество аренд
- суммарный доход

Упорядочить результат по суммарному доходу по убыванию, затем по названию жанра.

Ожидаемые столбцы

name · rentals_count · amount_sum

Задание 99

Задание будет открыто преподавателем на очной защите.

Лабораторная работа №3. Углубленные SQL-запросы

Описание базы данных

База данных описывает предметную область «Социальная сеть». Все задания лабораторной выполняются на уже созданной и заполненной базе.

- `users` — пользователи социальной сети
- `user_groups` — сообщества и группы
- `roles` — роли участников в группах
- `group_members` — членство пользователей в группах
- `posts` — публикации пользователей
- `comments` — комментарии к публикациям

Задание 1

Добавить новый комментарий к посту.

Значения:

- пользователь: `ivan.ivanov@example.com`
- группа: `Группа любителей кино`
- пост с датой создания `2023-01-01 12:00:00`
- текст комментария: `Очень интересная тема для обсуждения!`
- дата комментария: `2024-08-01 12:00:00`

Идентификаторы пользователя и поста нужно получить из базы.

Ожидаемые столбцы

`post_id` · `user_id` · `content` · `created_at`

Задание 2

Для пользователя `ivan.ivanov@example.com` в группе `Группа любителей кино` установить роль `Модератор`.

Ожидаемые столбцы

`member_id` · `role_id`

Задание 3

Вывести все посты группы `Группа любителей кино`, созданные начиная с `2023-01-01 00:00:00`.

Вернуть:

- название группы
- имя автора
- фамилию автора
- дату создания поста

- текст поста

Отсортировать по дате создания по убыванию.

Ожидаемые столбцы

group_name · first_name · last_name · created_at · content

Задание 4

Вывести группы, в которых в 2024 году не было ни одного поста.

Ожидаемые столбцы

name

Задание 5

Для каждой группы посчитать количество постов и среднее количество комментариев на один пост.

Ожидаемые столбцы

group_name · posts_count · avg_comments_per_post

Задание 6

Вывести группы, в которых состоит не менее трех пользователей.

Ожидаемые столбцы

group_name · members_count

Задание 7

Вывести email пользователей, которые оставляли комментарии к постам группы
Группа любителей кино .

Ожидаемые столбцы

email

Задание 8

Вывести группы с максимальным количеством участников.

Ожидаемые столбцы

group_name · members_count

Задание 9

Для каждой группы пронумеровать пользователей по количеству опубликованных постов в порядке убывания. Использовать RANK() .

Ожидаемые столбцы

group_name · email · posts_count · user_rank

Задание 10

Для каждой группы вывести по каждому посту накопительное количество постов в порядке даты публикации. Если дата публикации совпадает, порядок должен определяться по `post_id`.

Ожидаемые столбцы

`group_name` · `post_id` · `created_at` · `running_posts_count`

Задание 99

Задание будет открыто преподавателем на очной защите.

Лабораторная работа №4. Представления

Задание 1

Создать представление `basic_details_view`, которое возвращает детализированную информацию о публикациях.

Представление должно возвращать следующие столбцы:

- идентификатор публикации
- дата и время публикации
- название группы
- имя автора
- фамилия автора
- текст публикации

Представление должно быть read-only.

Ожидаемые столбцы

`post_id` · `created_at` · `group_name` · `author_first_name` · `author_last_name` · `content`

Задание 2

Создать обновляемое представление `filtered_entities_view`, которое показывает только группы, название которых начинается со слова `Группа`.

Представление должно возвращать следующие столбцы:

- идентификатор группы
- название группы
- описание группы
- дата создания

Представление должно поддерживать операции изменения данных и использовать `WITH CHECK OPTION`.

Ожидаемые столбцы

`group_id` · `name` · `description` · `created_at`

Задание 3

Создать представление `aggregated_stats_view`, которое для каждой группы показывает количество публикаций.

Представление должно возвращать следующие столбцы:

- идентификатор группы
- название группы
- количество публикаций

Ожидаемые столбцы

group_id · name · posts_count

Задание 4

Создать представление `advanced_aggregated_view`, которое для каждой группы показывает расширенную статистику.

Представление должно возвращать следующие столбцы:

- идентификатор группы
- название группы
- количество участников
- количество публикаций
- количество уникальных авторов публикаций

Ожидаемые столбцы

group_id · name · members_count · posts_count · authors_count

Задание 5

Создать представление `ranked_entities_view`, которое ранжирует пользователей по количеству публикаций (без повторения номера позиции в рейтинге).

Представление должно возвращать следующие столбцы:

- идентификатор пользователя
- имя
- фамилия
- количество публикаций
- позиция пользователя в рейтинге

Ожидаемые столбцы

user_id · first_name · last_name · posts_count · user_rank

Задание 6

Создать представление `running_totals_view`, которое для каждого пользователя показывает накопительный итог публикаций по датам.

Представление должно возвращать следующие столбцы:

- идентификатор пользователя
- идентификатор публикации
- дата и время публикации
- накопительное количество публикаций пользователя

Ожидаемые столбцы

user_id · post_id · created_at · running_posts_count

Задание 7

Создать представление `final_analytics_view` , которое для каждой группы показывает двух авторов с наибольшим числом публикаций в этой группе.

Представление должно возвращать следующие столбцы:

- идентификатор группы
- название группы
- идентификатор пользователя
- имя автора
- фамилия автора
- количество публикаций
- позиция автора внутри группы (с использованием `ROW_NUMBER`)

Для решения нужно использовать `CTE` и оконную функцию. В итоговом результате должны остаться только два автора с наибольшим числом публикаций внутри каждой группы. Если несколько авторов имеют одинаковое число публикаций - выбрать, упорядочив по `author_last_name` , `author_first_name` .

Ожидаемые столбцы

`group_id` · `group_name` · `user_id` · `author_first_name` · `author_last_name` · `posts_count` · `position_in_group`

Задание 99

Задание будет открыто преподавателем на очной защите.

Лабораторная работа №5. Процедуры, функции и триггеры

Задание 1

Создать функцию `business_metric_function`, которая по времени создания поста вычисляет количество полных дней, прошедших до текущей даты.

Функция должна:

- принимать время создания поста
- возвращать целое число дней
- корректно работать для уже существующих публикаций

Задание 2

Создать функцию `business_category_function`, которая по давности публикации возвращает ее категорию:

- `fresh`, если пост опубликован меньше 7 дней назад
- `active`, если пост опубликован от 7 до 30 дней назад включительно
- `archive`, если пост опубликован больше 30 дней назад

Функция должна вызывать функцию из задания 1.

Сигнатура функции

```
CREATE FUNCTION business_category_function(  
    target_created_at DATETIME  
)
```

Задание 3

Создать процедуру `report_procedure`, которая выводит отчет по публикациям в группах.

Отчет должен содержать:

- идентификатор поста
- группу
- автора
- время публикации
- возраст поста через `business_metric_function`
- категорию поста через `business_category_function`
- число комментариев к посту

Ожидаемые столбцы

```
post_id · group_name · first_name · last_name · created_at · post_age_days ·  
post_category · comments_count
```

Задание 4

Создать процедуру `action_procedure` , которая публикует новый пост в группе.

Параметры процедуры:

- email пользователя
- название группы
- текст публикации
- дату и время публикации

Логика процедуры:

- найти `user_id` и `group_id` по бизнес-атрибутам
- создать запись в таблице `posts`

Это должна быть одна локальная операция публикации поста.

Задание 5

Создать триггер `autofill_trigger` , который автоматически подставляет корректное время публикации при вставке записи.

Правило:

- если при вставке `created_at` не задано, нужно подставить текущие дату и время

Задание 6

Создать триггер `validation_trigger` , который запрещает публиковать пост пользователю, не состоящему в выбранной группе.

Правило:

- нельзя вставлять новую запись в `posts` , если для пары `group_id + user_id` нет записи в `group_members`

При нарушении правила триггер должен завершать операцию с понятным сообщением об ошибке.

Задание 7

Создать триггер `audit_trigger` , который журналирует удаление записи в таблице `posts` и сохраняет запись в таблице `post_audit_log` . Таблица уже присутствует в схеме БД.

Триггер должен сохранять в таблицу журнала запись с меткой `DELETE` (значение поля `operation_note`).

Задание 8

Создать процедуру `workflow_procedure` , которая удаляет пост автора, если под ним еще нет комментариев.

Параметры процедуры:

- идентификатор поста
- email автора

Логика процедуры:

- проверить, что такой пост существует
- проверить, что указанный пользователь действительно является автором поста
- проверить, что у поста нет комментариев
- удалить пост

Задание 99

Задание будет открыто преподавателем на очной защите.

Лабораторная работа №6. Индексы, транзакции и JSON

Задание 1

Написать запрос, который выводит историю публикаций группы `group_id = 1`, созданных не раньше `2023-01-01`.

Ожидаемые столбцы

`post_id` · `group_id` · `user_id` · `created_at`

Задание 2

Создать составной индекс, ускоряющий выполнение запроса из задания 1.

Ожидаемые столбцы

`post_id` · `group_id` · `user_id` · `created_at`

Задание 3

Написать запрос, который соединяет `posts` и `comments` и выводит комментарии к публикациям, созданным после `2023-01-01`.

Ожидаемые столбцы

`post_id` · `created_at` · `comment_id` · `user_id`

Задание 4

Создать индекс в таблице `comments`, ускоряющий выполнение запроса из задания 3.

Ожидаемые столбцы

`post_id` · `created_at` · `comment_id` · `user_id`

Задание 5

Написать запрос, который выводит группы, у которых в JSON-поле `metadata` по ключу `location` хранится значение `Москва`.

Ожидаемые столбцы

`group_id` · `name` · `location_info`

Задание 6

Создать функциональный индекс в таблице `user_groups`, ускоряющий выполнение запроса из задания 5.

Ожидаемые столбцы

`group_id` · `name` · `location_info`

Задание 7

Создать процедуру `atomic_post_procedure`, которая в одной транзакции публикует новый пост: добавляет запись в `posts` и связанную запись в техническую таблицу `post_operation_log` со значением `operation_note = 'created'`. Если вставка в `post_operation_log` не удалась, запись в `posts` должна быть отменена. При принудительной ошибке процедура должна завершаться ошибкой для вызывающей стороны (исключение не должно подавляться).

Параметры процедуры:

- `p_group_id` - идентификатор группы
- `p_user_id` - идентификатор пользователя
- `p_force_error` - если `TRUE`, при добавлении связанной записи установить `operation_note = NULL`, выполнить `ROLLBACK` и пробросить ошибку наружу (`RESIGNAL`)

Сигнатура процедуры

```
CREATE PROCEDURE atomic_post_procedure(  
  IN p_group_id INT,  
  IN p_user_id INT,  
  IN p_force_error BOOLEAN  
)
```

Задание 8

Создать процедуру `savepoint_post_procedure`, которая выполняет пакетную публикацию постов: сохраняет первый корректный пост в `posts` и связанную запись в `post_operation_log`, затем после `SAVEPOINT` пытается выполнить вторую вставку и откатывает только эту часть операции при ошибке.

Параметры процедуры:

- `p_group_id` - идентификатор группы
- `p_user_id` - идентификатор пользователя
- `p_force_second_error` - если `TRUE`, второй шаг должен гарантированно завершиться ошибкой

Сигнатура

```
CREATE PROCEDURE savepoint_post_procedure(  
  IN p_group_id INT,  
  IN p_user_id INT,  
  IN p_force_second_error BOOLEAN  
)
```

Задание 9

Продemonстрировать возможность чтения незафиксированных изменений при использовании уровня изоляции `READ UNCOMMITTED` и отсутствие такой возможности при уровне изоляции `READ COMMITTED` на примере обновления публикации без `COMMIT`.

Задание 10

Продemonстрировать возможность изменения результата повторного чтения одной и той же записи при использовании уровня изоляции `READ COMMITTED` и отсутствие такого эффекта при уровне изоляции `REPEATABLE READ` на примере обновления публикации.

Лабораторная работа №7. Введение в NoSQL на примере MongoDB

Описание базы данных

Контекст домена

LP7 использует ту же предметную область, что и SQL-лабораторные работы, но представляет данные в MongoDB в виде документов.

Подготовленные системой коллекции

- `post_documents`

Основная коллекция

Коллекция `post_documents` содержит документы о публикациях в группах.

Обязательные поля документа:

- `postCode`
- `visibility`
- `createdAt`
- `author` — вложенный объект
- `group` — вложенный объект
- `tags` — массив тегов
- `comments` — массив комментариев

Поля для типовых запросов

- простое поле: `visibility`
- вложенное поле: `author.email`
- массив: `tags`
- аналитика: `group.title` , `group.category` , `createdAt`

Подготовленные системой данные

Система заранее создает коллекцию `post_documents` , валидатор и контрольные документы `POST-1001` и `POST-1002` .

Структура вложенных объектов и массивов

- `author` : `email` , `fullName`
- `group` : `groupCode` , `title` , `category`
- элемент `tags` : строковый тег
- элемент `comments` : `authorEmail` , `text` , `createdAt`

Исходное состояние контрольных документов

- документ `POST-1001` уже содержит массив `comments` , в который можно добавлять новый комментарий
- документ `POST-1002` существует в режиме видимости, который можно перевести в `private` без нарушения схемы

Общая логика варианта

Вариант построен вокруг основной сущности домена - публикации, представленной в MongoDB как документ.

- задание 1 добавляет новый документ
- задание 2 добавляет новую вложенную структуру
- задания 3–5 проверяют выборки по простому полю, вложенному полю и массиву
- задание 6 изменяет существующий документ
- задание 7 строит aggregation pipeline
- задание 8 создает MongoDB view на новом pipeline

Задание 1

Добавить новый документ в коллекцию `post_documents` с кодом `POST-9001`.

Документ должен содержать следующие поля и структуры:

- `postCode`: "POST-9001"
- `visibility`: "public"
- `createdAt`: `ISODate("2024-07-19T14:00:00Z")`
- `author` :
 - `email`: "olga@example.com"
 - `fullName`: "Ольга Соколова"
- `group` :
 - `groupCode`: "GR-01"
 - `title`: "Путешествия"
 - `category`: "Хобби"
- `tags` содержит элементы:
 - "лето"
 - "море"
- `comments` содержит элемент:
 - `authorEmail`: "ivan@example.com"
 - `text`: "Отличный пост"
 - `createdAt`: `ISODate("2024-07-19T15:00:00Z")`

Задание 2

Добавить новый комментарий в массив `comments` документа `POST-1001`.

В массив `comments` нужно добавить элемент со следующими данными:

- `authorEmail`: "maria@example.com"
- `text`: "Интересно!"
- `createdAt`: `ISODate("2024-07-20T09:00:00Z")`

Задание 3

Написать запрос, который выводит публичные публикации.

Ожидаемые поля

`postCode` · `visibility` · `createdAt` · `group.title`

Задание 4

Написать запрос, который находит публикации автора по email `olga@example.com` .

Ожидаемые поля

`postCode` · `author.fullName` · `visibility`

Задание 5

Написать запрос, который выводит публикации с тегом `лето` .

Ожидаемые поля

`postCode` · `tags` · `visibility`

Задание 6

Обновить видимость публикации `POST-1002` , переведя её в `private` .

Нужно изменить следующие поля:

- `visibility: "private"`
- `"group.category": "Архив"`

Задание 7

Построить aggregation pipeline, который учитывает только публикации с видимостью `public` и `private` , группирует их по группам и для каждой группы возвращает количество публикаций и дату последней публикации.

Результат агрегации должен содержать поля, перечисленные в разделе `Ожидаемые поля` .

Ожидаемые поля

`_id` · `postsCount` · `lastPostDate`

Задание 8

Создать MongoDB view `popular_tags_view` , которое содержит сводку по тегам публикаций.

В представление должны попадать все документы публикаций из коллекции `post_documents` . Для каждого тега нужно вернуть:

- `tag` — название тега
- `postsCount` — количество публикаций, в которых встречается тег

Ожидаемые поля

tag · postsCount

Задание 99

Задание будет открыто преподавателем на очной защите.

Экзамен по дисциплине

Задание 1

Вывести посты пользователя `Ивана Иванова` , опубликованные в группе `Группа любителей кино` .

В запросе вернуть следующие столбцы:

- идентификатор поста
- содержимое поста
- дата создания поста
- имя пользователя
- фамилия пользователя

Результат упорядочить по фамилии, затем по имени пользователя.

Ожидаемые столбцы

`post_id` · `content` · `created_at` · `first_name` · `last_name`

Задание 2

Для каждого пользователя подсчитать количество его постов.

Включить в результат всех пользователей, в том числе тех, у кого нет ни одного поста.

В запросе вернуть следующие столбцы:

- идентификатор пользователя
- имя пользователя
- фамилия пользователя
- количество постов

Ожидаемые столбцы

`user_id` · `first_name` · `last_name` · `posts_count`

Задание 3

Вывести пользователей, у которых не менее трёх постов в группе `Спортивный клуб "Энергия"` .

В запросе вернуть следующие столбцы:

- идентификатор пользователя
- имя пользователя
- фамилия пользователя
- количество постов в данной группе

Ожидаемые столбцы

`user_id` · `first_name` · `last_name` · `posts_count`

Задание 4

Вывести пользователей, у которых количество постов превышает среднее количество постов по всем пользователям.

В запросе вернуть следующие столбцы:

- идентификатор пользователя
- имя пользователя
- фамилия пользователя
- количество постов

Ожидаемые столбцы

`user_id` · `first_name` · `last_name` · `posts_count`

Задание 5

Для каждой группы проранжировать пользователей по убыванию количества их постов в этой группе.

Требования к рангу:

- ранг считается отдельно внутри каждой группы
- у каждого пользователя свой уникальный ранг — повторений быть не должно даже при одинаковом количестве постов
- при равном количестве постов упорядочивать по фамилии пользователя

В запросе вернуть следующие столбцы:

- название группы
- имя пользователя
- фамилия пользователя
- количество постов
- ранг пользователя внутри группы

Ожидаемые столбцы

`group_name` · `first_name` · `last_name` · `posts_count` · `rank_in_group`

Задание 6

Создать представление `exam_view`, которое для каждого пользователя показывает суммарное количество его постов и ранг среди всех пользователей по убыванию числа постов.

Использовать оконную функцию `RANK()`.

Представление должно возвращать следующие столбцы:

- идентификатор пользователя
- имя пользователя
- фамилия пользователя

- суммарное количество постов
- ранг пользователя по количеству постов (1 — у кого больше всего)

Ожидаемые столбцы

user_id · first_name · last_name · total_posts · rank_by_posts

Задание 7

Создать функцию exam_function , которая принимает идентификатор пользователя, подсчитывает количество его постов и возвращает категорию активности:

- высокая активность — 10 постов и более
- средняя активность — от 3 до 9 включительно
- низкая активность — менее 3

Задание 8

Создать хранимую процедуру exam_procedure , которая принимает количество постов и возвращает пользователей, у которых постов **больше** заданного числа.

Результат должен содержать следующие столбцы:

- идентификатор пользователя
- имя пользователя
- фамилия пользователя
- количество постов

Ожидаемые столбцы

user_id · first_name · last_name · posts_count